

Asynchronous programming

Future, async and await

Eng | Ruqaih Hussein Salman

2025-2026

Overview

Unlike Android, which is multi-threaded, i.e. any task can be assigned to different threads and can be executed simultaneously, Dart/Flutter is single-threaded, that simply means we can't have 2 or more threads run parallel. Only a single thread picks up event from the event queue and processes them.

In this lecture, we will familiarize ourselves with the concept of **Asynchronous programming**, which many developers find it tricky to understand at first place and how dart uses API's like **Future**, **async** and **await** to implement this. But first, let's understand what synchronous programming is.

Synchronous Programming

Synchronous programming is a form of programming in which program executes line by line. When a function is called, program execution will wait until the function returns. Let's understand this with an example

Suppose there are 3 methods, that needs to be executed

method1()
method2()
method3()

With synchronous programming, methods will execute one by one, in the above-given sequence. It doesn't matter, what functions are returning or how long they'll take to execute its body.

```
void main() {  
  method1();  
  method2();  
  method3();  
}  
void method1() {  
  print('I\'m the body of method 1');  
}  
void method3() {  
  print('I\'m the body of method 3');  
}  
void method2() {  
  print('I\'m the body of method 2');  
}
```

In the above example, method1() executes first and then method2() and method3().

Now suppose **method2()** performs some operation like fetching data from a huge database or downloading an image. Then it might take a few seconds to execute such long-heavy tasks. And till the

time **method2()** is executing no other subsequent function in the '**Function Stack**' can be executed. Since with Synchronous programming, we don't get such flexibility.

This can freeze the UI of our application and results in crashing of the application and bad UX ` (**Long-heavy** operation here is a term used to indicate tasks, which can't execute instantly and can take a few seconds to execute. E.g — Network operations, Image processing, file downloading etc.

Now you might be wondering how to solve such problem and improve the performance of our application, as dart doesn't support the concept of multiple threads.

Asynchronous Programming

Asynchronous programming is a form of parallel programming which allows execution of events parallelly with a single thread which has control over all these events. Dart uses API's like **Future**, **async** and **await** to implement this.

Why asynchronous code matters

Asynchronous operations let your program complete work while waiting for another operation to finish. Here are some common asynchronous operations:

- Fetching data over a network.
- Writing to a database.
- Reading data from a file.

Let's take one more example and understand this concept thoroughly.

Again suppose there are 3 methods, that needs to be executed

method1() -> Normal function

method2(callback) -> Some resource-heavy function

method3() -> Normal function

Here **method2()** is a **long-heavy** function and has a callback which means compiler won't stop here and execute subsequent methods until **method2()** return some data or error. Now you might ask how **method2()** is different from other methods and how it behaves differently.

Yeah right. How?

Here **method2()** will return an object of **Future** class which is similar to **Promises** available in other programming languages. In simple words, Future is a promise to the compiler, that it'll give him something in return once it has something to give. Did that go a little overboard? No worries `).

Suppose **method2()** is returning some data from a huge database. This task might take a few seconds to execute. Here we'll use **Future** that tells the compiler "**Don't wait for me to execute and return something, instead go and execute some other functions. I'll give you the data/error once I have that with me.**"

By definition Future object has three states-

- **Uncompleted** — When Future object is still incomplete.
- **Completed with data** — When Future object is completed and get its data.
- **Completed with an error** — When Future object is completed but get an error.

Now understand the concept with some code snippet.

Now understand the concept with some code snippet.



```
void main() {  
  method1();  
  method2();  
  method3();  
}  
  
// Some normal functions  
  
void method1() {  
  print('I'm the body of method 1');  
}  
  
void method3() {  
  print('I'm the body of method 3');  
}  
  
//Some resource heavy function  
  
void method2() {  
  Future.delayed(Duration(seconds: 5), () {  
    print('I'm the body of method 2');  
  });  
}
```

I'm the body of method 1
I'm the body of method 3
I'm the body of method 2

Executes after 5 secs

Documentation

Here, since **method2()** is taking 5 seconds to execute. In the meantime **method1()** and **method3()** get execute. Here we have used the most basic form of **Future.delayed()** which isn't returning anything.

Now let's take one more scenario where our **method2()** is returning some Future value, where we are returning a **Future** Object of **String** type.

```
void main() {  
  method1();  
  Future<String> content = method2();  
  print('I\'m body of method 2. $content');  
  method3();  
}  
  
// Some normal functions  
  
void method1() {  
  print('I\'m the body of method 1');  
}  
  
void method3() {  
  print('I\'m the body of method 3');  
}  
  
//Some resource heavy function that is returning a Future object  
  
Future<String> method2() {  
  Future<String> resultantData = Future.delayed(Duration(seconds:  
5), () {  
    return 'I need 5 seconds to execute';  
  });  
  return resultantData;  
}
```

Console

```
I'm the body of method 1  
I'm body of method 2. Instance of '_Future<String>'  
I'm the body of method 3
```

Documentation

Notice the unexpected output, this is because at the time when our program started we didn't have a value of **resultantData**. We just had a valid Future object, but we are getting the value of **resultantData** after the specified delay time which means that when the program ended then only we would have got the value of **resultantData**.

Now how to get the expected output?

For getting the value of the **content** variable we need to use the term **await** and **async**.

```
import 'dart:async';  
void main() async {  
  method1();  
  String content = await method2();  
  print('I\'m body of method 2. $content');  
  method3();  
}  
  
// Some normal functions  
  
void method1() {  
  print('I\'m the body of method 1');  
}  
  
void method3() {  
  print('I\'m the body of method 3');  
}  
  
//Some resource heavy function that is returning a Future object  
  
Future<String> method2() {  
  Future<String> resultantData = Future.delayed(Duration(seconds: 5),  
() {  
    return 'I need 5 seconds to execute';  
  });  
  return resultantData;  
}
```

Console

```
I'm the body of method 1  
I'm body of method 2. I need 5 seconds to execute  
I'm the body of method 3
```

Executes after 5 seconds

Documentation

Once a compiler encounters an **await** keyword then it'll only execute the code further when it gets a value/error and now since we are getting a valid value for **content**, we'll change it's type to its original type,

i.e. String. Here when compiler encountered an **await** keyword. It'll pause the execution of code further and only when it gets the value for content, it'll execute the code further.

Note: async and await are keywords that provide a way to make asynchronous operations appear synchronous.

A synch Flutter widget

Since the **build** method in Flutter runs synchronously, we need to find a way to ensure that the app will build widgets based on the data that will be received in the future.

Stateful Widget

One approach is to use **StatefulWidget** and set the state while information is obtained:

➤ Example:

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() {
  runApp(MaterialApp(home: MyFutureWidget()));
}

Future<String> fetchName() async {
  final Uri uri = Uri.https('Eng: Ruqaih Salman.com', '/getRandomName');
  final http.Response name = await http.get(uri);
  return jsonDecode(name.body);
}

class MyFutureWidget extends StatefulWidget {
  @override
  _MyFutureWidgetState createState() => _MyFutureWidgetState();
}

class _MyFutureWidgetState extends State<MyFutureWidget> {
  String? value;

  @override
  void initState() {
    super.initState();

    // fetchName function is a asynchronously to GET http data
    fetchName().then((result) {
      // Once we receive our name we trigger rebuild.
      setState(() {
        value = result;
      });
    });
  }

  @override
  Widget build(BuildContext context) {
```

```
// When value is null show loading indicator.
if (value == null) {
  return Center(
    child: Container(
      height: 100,
      width: 130,
      child: const CircularProgressIndicator(
        strokeWidth: 10,
      )),
  );
}
return Text('Fetched value: $value');
}
```

output:

In this example, you may have noticed that we didn't properly handle possible exceptions, which we can solve by adding an error variable. The process above will work, but we can improve on it.



FutureBuilder widget

FutureBuilder provides a cleaner, better way to handle future in Flutter. **FutureBuilder** accepts a **future** and builds a widget when the data is resolved:

```
const FutureBuilder({
  Key? key,
  this.future,
  this.initialData,
  required this.builder,
}) : assert(builder != null), super(key: key);
```

Let's take a closer look at how the **FutureBuilder** widget works:

```
FutureBuilder<String>(
  future: FUTURE,
  initialData: null,
  builder: (BuildContext context, AsyncSnapshot<String> snapshot) {
  }
);
```

The second parameter in the **build** function is a type of **AsyncSnapshot** with a specified data type. For example, in the code above, we have defined **String**.

The snapshot is an immutable representation of the most recent interaction with an asynchronous computation. It has several properties. When an asynchronous computation occurs, it is beneficial to know the state of the current connection, which is possible via **snapshot.connectionState**.

The **connectionState** has four usual flows:

- **none**: maybe with some initial data
- **waiting**: asynchronous operation has begun. The data is typically null
- **active**: data is non-null and has the potential to change over time
- **done**: data is non-null

snapshot.data returns the latest data, and **snapshot.error** returns the newest error object. **snapshot.hasData** and **snapshot.hasError** are two handy getters that check whether an error or data have been received. **FutureBuilder** is a **StatefulWidget** that uses state as a snapshot.

➤ Example: FutureBuilder with issue

A FutureBuilder widget builds itself based on the state of a given future. For this example, let's assume you have a function called `getData()` that returns a `Future<String>`. Many developers start their experimentation with this widget using code something like this:

```
class MyStatelessWidget extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return FutureBuilder(
      future: getData(),
      builder: (BuildContext context, AsyncSnapshot snapshot) {
        if (snapshot.connectionState == ConnectionState.waiting) {
          return CircularProgressIndicator();
        }

        if (snapshot.hasData) {
          return Text(snapshot.data);
        }
        return Container();
      },
    );
  }
}
```

This custom stateless widget returns a `FutureBuilder` that will display a progress indicator if the future returned by `getData()` has not yet completed, and it will show the data if the future has completed with a value. If neither of those things is true, an empty `Container` is rendered instead. You tell the `FutureBuilder` which future to watch with its `future` parameter, then give it a builder function that will be called for every rebuild. The builder callback receives the usual `BuildContext` argument common to all Flutter build operations, and it also takes an instance of `AsyncSnapshot`, which you can use to check the future's status and retrieve any data.

There is a problem with this approach. According to the official documentation for `FutureBuilder`, the provided future needs to have been obtained prior to the build step. Otherwise the asynchronous call for data will be repeatedly executed, essentially starting over with every build. Flutter widgets can be rebuilt at any time for many reasons, including animation or user interaction, so the builder function provided to `FutureBuilder` could execute many times in a single second. Normally you don't want to retrieve the same data again and again, so you need to make the retrieval call outside the `FutureBuilder`.

To fix it,

Notice that I used the `getName()` function directly in my **`FutureBuilder`** inside the `build` method. Every time the **`FutureBuilder`**'s parent is rebuilt, the asynchronous task will be restarted, which is not good practice. Solve this problem by moving the `future` to be obtained as early as possible – for example, during `initState` on a **`StatefulWidget`**:

```
class _MyStatefulWidgetState extends State<MyStatefulWidget> {
  Future<String> _dataFuture;

  @override
  void initState() {
    super.initState();

    _dataFuture = getData();
  }

  @override
  Widget build(BuildContext context) {
    return FutureBuilder(
      future: _dataFuture,
      builder: (BuildContext context, AsyncSnapshot snapshot) {
        if (snapshot.connectionState == ConnectionState.waiting) {
          return CircularProgressIndicator();
        }

        if (snapshot.hasData) {
          return Text(snapshot.data);
        }

        return Container();
      },
    );
  }
}
```

This version of the widget acquires the data future during `initState()`. The `initState()` method will be called exactly once when the widget's state object is created. It will not be executed during a widget rebuild. The future is stored in a private member variable, then provided to `FutureBuilder`. The builder function hasn't changed at all.

➤ Example:

```
import 'package:flutter/material.dart';
void main() => runApp(FuturBuilder_Roq_Ex());
class FuturBuilder_Roq_Ex extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'FuturBuilder Roq App',
      // to hide debug banner
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        primarySwatch: Colors.green,
      ),
      home: HomePage_FuturBuilder_Roq_Ex(),
    );
  }
}

class HomePage_FuturBuilder_Roq_Ex extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return SafeArea(
      child: Scaffold(
        appBar: AppBar(
          title: Text('FuturBuilder Roq App'),
        ),
        body: Center(
          child: ElevatedButton(
            onPressed: () => Navigator.push(
              context,
              MaterialPageRoute(
                builder: (ctx) => FutureDemoPage(),
              ),
            ),
          ),
          child: Text('Demonstrate FutureBuilder By D.Ruqaih Salman'),
        ),
      ),
    );
  }
}

class FutureDemoPage extends StatelessWidget {
  /// Function that will return a "string" after some time To demonstrate network call
  /// delay of [2 seconds] is used This function will behave as an
  /// asynchronous function
  Future<String> getData() {
    return Future.delayed(Duration(seconds: 5), () {
      return "I am the data that appeared after five seconds";
    });
  }
}

@override
```

```

Widget build(BuildContext context) {
  return SafeArea(
    child: Scaffold(
      appBar: AppBar(
        title: Text('Future RoqNew Page'),
      ),
      body: FutureBuilder(
        builder: (ctx, snapshot) {
          // Checking if future is resolved or not
          if (snapshot.connectionState == ConnectionState.done) {
            // If we got an error
            if (snapshot.hasError) {
              return Center(
                child: Text(
                  '${snapshot.error} occured',
                  style: TextStyle(fontSize: 18),
                ),
              );

              // if we got our data
            } else if (snapshot.hasData) {
              // Extracting data from snapshot object
              final String data_roq = snapshot.data as String;
              return Center(
                child: Text(
                  '$data_roq',
                  style: TextStyle(fontSize: 18),
                ),
              );
            }
          }

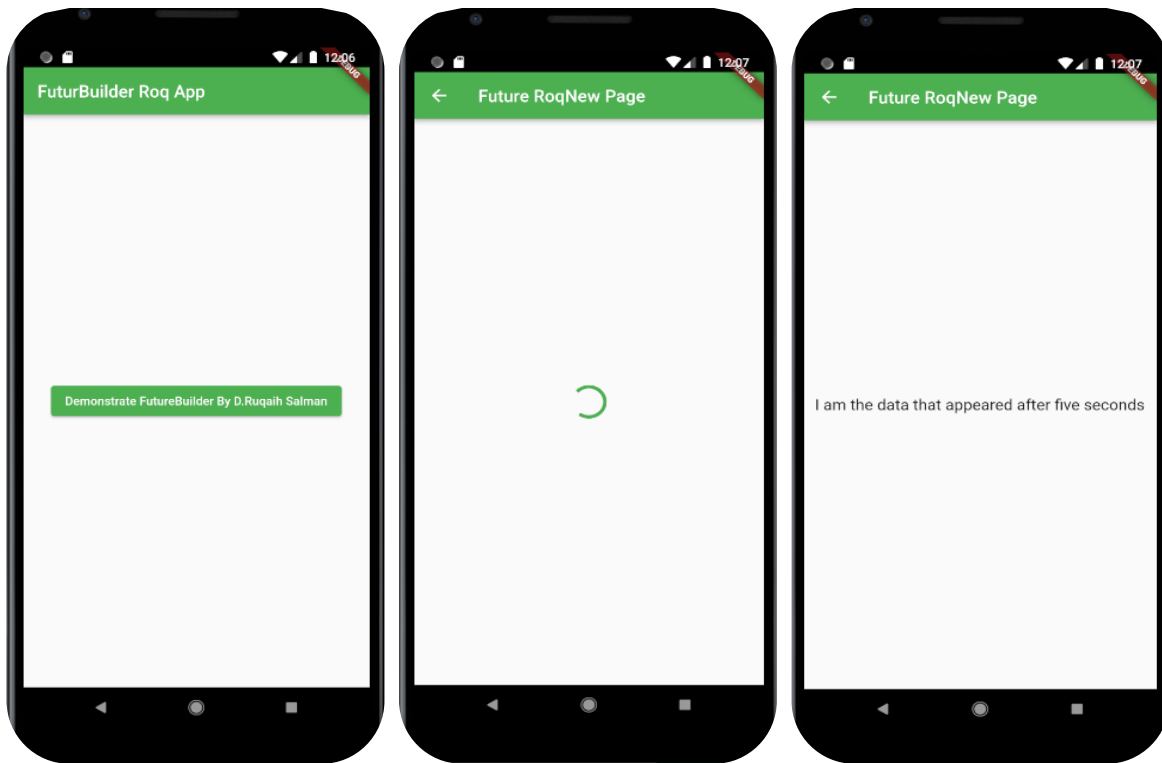
          // Displaying LoadingSpinner to indicate waiting state
          return Center(
            child: CircularProgressIndicator(),
          );
        },

        // Future that needs to be resolved
        // inorder to display something on the Canvas
        future: getData(),
      ),
    ),
  );
}

```

Output

When we run the app in Android Studio, we can see the following screens in our Emulator.



StreamBuilder widget

It's worth also taking a look at StreamBuilder, another widget that handles stream. StreamBuilder and FutureBuilder are nearly identical. However, StreamBuilder delivers data periodically, so you have to listen to it more frequently than FutureBuilder, which you must listen to only once. StreamBuilder is a widget that builds itself based on the latest snapshot of interaction with a stream. The StreamBuilder widget automatically subscribes and unsubscribes from the stream. When disposing a widget, you don't have to worry about unsubscribing, which could cause a memory leak:

```
@override
Widget build(BuildContext context) {
  return StreamBuilder<String>(
    stream: dataStream,
    builder: (BuildContext context, AsyncSnapshot<String> snapshot) {
      },
  );
}
```

Why StreamBuilder widget

The **StreamBuilder** widget is used in many kinds of Flutter applications, especially chat applications, social networks, real-time content updates, etc. In this article, we will go over 2 complete examples of implementing StreamBuilder: the first example is a **real-time clock app**, and the second one is a **demo chat app**. But first, let's explore the fundamentals of the widget.

Main arguments:

- **initialData**: The initial data that will be used to create the initial snapshot.
- **builder**: The build strategy currently used by this builder.
- **stream**: The asynchronous computation to which this builder is currently connected, possibly null. When changed, the current summary is updated.

Understanding `async*` and `yield`

In the following example, you will notice these keywords: **`async*`** and **`yield`**. They are all keywords used in **generator functions**.

A **generator function** is a function that produces a sequence of values (in contrast to regular functions that return a single value) and is often used with **Stream**.

Yield is a keyword that “returns” a single value to the sequence but does not stop the generator function.

➤ Example : Real-time Clock App

```
import 'package:flutter/material.dart';
import 'dart:async';

void main() {
  runApp(const Clock_Roq());
}

class Clock_Roq extends StatelessWidget {
  const Clock_Roq({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      // Remove the debug banner
      debugShowCheckedModeBanner: false,
      title: 'ÓRoqSal App',
      theme: ThemeData(
        // enable Material 3
        useMaterial3: true,
        primarySwatch: Colors.blue,
      ),
      home: const HomePage(),
    );
  }
}

class HomePage extends StatelessWidget {
  const HomePage({Key? key}) : super(key: key);

  final bool _running = true;

  Stream<String> _clock() async* {
    // This loop will run forever because _running is always true
    while (_running) {
      await Future<void>.delayed(const Duration(seconds: 1));
      DateTime now = DateTime.now();
      // This will be displayed on the screen as current time
    }
  }
}
```

```

        yield "${now.hour} : ${now.minute} : ${now.second}";
    }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Clock App '),
    ),
    body: Center(
      child: StreamBuilder(
        stream: _clock(),
        builder: (context, AsyncSnapshot<String> snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const CircularProgressIndicator();
          }
          return Text(
            snapshot.data!,
            style: const TextStyle(fontSize: 50, color: Colors.blue),
          );
        },
      ),
    ),
  );
}

```

output:

This app displays a real-time clock in the center of the screen.



